
**by Kunal Sir**

React Context API Assignment

---

# Part A – Multiple Choice Questions

---

1. The React application starts execution from which file?
   a) App.js b) Index.js c) Main.js d) Home.js

---

2. The `render()` method in React determines:
   a) What CSS to load b) Which component to render and where c) The routing
   d) None of these

---

3. In React, the component included inside another component is called a:
   a) Root component b) Child component c) Parent component d) Subcomponent

---

4. The component that renders another component is known as:
   a) Child component b) Parent component c) Nested component d) Context provider

---

5. The problem with props drilling occurs when:
   a) Data is passed to unrelated components b) Data is deeply nested across many child components c) Components have no state d) We use hooks

---

6. The Context API helps to:
   a) Store large files  b) Pass data between non-related components easily
   c) Manage databases  d) Optimize network requests

---

7. The function used to create a context in React is:
   a) create()  b) createContext()  c) makeContext()  d) useContext()

---

8. To consume or access data from Context in child components, we use:
   a) getContext()  b) useContext()  c) setContext()  d) contextData()

---

9. To provide data to child components through Context, we use:
   a) Provider component  b) Consumer component  c) Renderer component
   d) Props component

---

10. The Context Provider passes data using which attribute?
    a) data  b) info  c) value  d) context

---

11. In Context API, which hook is required in the child component?
    a) useState  b) useContext  c) useEffect  d) useReducer

---

12. The Context API removes the need for:
    a) Hooks  b) Props drilling  c) Components  d) States

---

13. To make Context data available to children, we must wrap them with:
    a) <App>  b) <Provider>  c) <Context>  d) <Wrapper>

---

14. The `useContext()` hook accepts which argument?
    a) The component name  b) The created context object  c) The provider name
    d) The root element ID

---

15. In which file do we usually wrap the Provider component?
    a) App.js  b) Demo.js  c) Child.js  d) public/index.html

---

16. The Context API is mainly used for:
    a) Styling  b) Data sharing between components  c) Routing  d) Animation

---

17. What should be exported from the parent to be used in children for accessing data?
    a) The state  b) The context  c) The provider  d) The prop

---

18. The Context API internally uses which feature of React?
    a) Virtual DOM  b) Hook mechanism  c) Component tree  d) JavaScript classes

---

19. When using Context API, the data flow is:
    a) Random  b) Global and accessible across nested components  c) One-way only
    d) Circular

---

20. Context API helps maintain:
    a) Repetition of code  b) Centralized data access  c) Heavy rendering
    d) Unused states

---

# Part B – Problem Statements

---

1. **Basic Context Setup**
   Create a new React project and make a `MessageContext`.
   In your `App.js`, use `createContext()` to store a message like `"Welcome to React Context API"`.
   Use a `MessageProvider` to pass this message to a `Child` component and display it using `useContext()`.
   (Objective: Understand how to create and use a simple Context.)

---

2. **Sharing Data Between Components**
   Create two components – `Parent` and `Child`.
   In the parent, define a context that holds a student's name (e.g., `"Sakshi"`).
   Use Context API to send this data from the parent to the child component and display `"Student Name: Sakshi"` inside the child.
   (Objective: Learn how Context helps share data without props.)

---

3. **Using Context to Change Style**
   Make a `ColorContext` that stores a color value (like `"green"`).
   The parent provides the color, and the child component displays a heading whose text color is taken from the context.
   Try changing the color value in the context and see how the child automatically reflects the new color.
   (Objective: Understand re-rendering behavior with context values.)

---

4. **Accessing Context in Multiple Components**
   Create a `UserContext` that stores `{ name: "Rohit", age: 21 }`.
   Have three components: `App`, `Profile`, and `Details`.
   Provide the context from `App`, and consume it in both `Profile` and `Details` to display the user's name and age.
   (Objective: Learn that multiple components can use the same Context.)

---

5. **Light and Dark Theme Using Context**
   Create a `ThemeContext` with the value `"light"`.
   In `App.js`, wrap two components – `Header` and `Footer` – with the `ThemeProvider`.
   Display a background color according to the current theme in both components.
   Change the theme value to `"dark"` and see both components update automatically.
   (Objective: Understand how Context creates a shared theme across the app.)

---

6. **Counter Application Using Context API**
   Make a context called `CounterContext` that stores `count` and a method `setCount`.
   Use `useState()` in the parent component to manage the count, and pass both through the provider.
   Create two child components: `IncrementButton` and `DecrementButton`.
   Each should access and update the same count value via context.
   (Objective: Learn how to share both data and functions using context.)

---

7. **Language Selector Context**
   Create a `LanguageContext` that stores the currently selected language (`"English"`, `"Hindi"`, or `"Marathi"`).
   The parent component will provide the language, and two child components (`Greeting` and `About`) will consume it to display translated text based on the selected language.
   (Objective: Demonstrate real-world data sharing like language translation.)

---

8. **Login System with Context**
   Create a `LoginContext` that manages user login status using a boolean value `isLoggedIn`.
   In the `App` component, display two buttons – "Login" and "Logout" – which toggle the state.
   Another component, `Navbar`, should show either "Welcome, User" or "Please Login" based on the context.
   (Objective: Practice global state management through Context.)

---

9. **Font Size Controller Using Context**
   Create a `FontContext` that stores a font size value (e.g., `"18px"`).
   The parent provides this size, and the `Paragraph` component displays text using it.
   Add two buttons in another component to increase or decrease the font size, updating it through the context.
   (Objective: Learn how to update and reflect context data dynamically.)

---

10. **App Info Display Using Context API**
    Create an `AppInfoContext` that stores `{ appName: "Student Portal", version: "1.0" }`.
    The parent component (`App`) provides this context, and a `Footer` component consumes it to display something like:
    "Application: Student Portal | Version: 1.0".
    (Objective: Reinforce how to consume static data across components using context.)

CJC

Complete Java Classes

**by Kunal Sir**



CJC

Complete Java Classes

Stop, Near, 1st Floor, Above Rupam Sweets/ Priyanka Collections Building Vikas Mitra Mandal Chowk Road, Karve Nagar, Pune, Maharashtra 411052 , Mobile No.- 8888022204